Davante

19

PROGRAMACIÓN

Base de datos objeto-relacional

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Base de datos objeto-relacional 5

/ 3. ORM 6

/ 4. Caso práctico 1: "¿Cuándo emplear Hibernate?" 7

/ 5. CRUD 7

/ 6. Objeto de acceso a datos: DAO 8

/ 7. Ejecución de sentencias: CRUD 9

7.1. Create() 9

7.2. Read() 10

7.3. Update() 11

7.4. Delete() 12

/ 8. Movimiento y posicionamiento del cursor ResultSet 13

8.1. Movimiento del cursor 13

8.2. Ubicación del cursor 14

/ 9. SQL injection 14

/ 10. Sentencias parametrizadas 15

/ 11. Caso práctico 2: "Inyección SQL" 16

ÍNDICE

/ 12. Bases de datos e interfaz gráfica 16

/ 13. Modelo-Vista-Controlador 17

13.1. Interacción MVC 18

/ 14. Resumen y resolución del caso práctico de la unidad 18

/ 15. Bibliografía 19

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Realizar el mapeo objeto-relacional de clases.

Desarrollar programas con acceso a bases de datos relacionales.

Insertar datos en una base de datos relacional.

Recuperar información de una base de datos relacional.

Borrar y modificar datos de una base de datos relacional.

Acceder a los resultados de las distintas operaciones realizadas en la base de datos relacional.

Conocer las vulnerabilidades de seguridad mediante inyección SQL y reducirla mediante sentencias parametrizadas.

/ 1. Introducción y contextualización práctica

El mapeo objeto-relacional permite integrar clases de un programa con tablas de bases de datos relacionales. Es crucial para desarrollar aplicaciones que interactúen con datos almacenados de forma estructurada. Se pueden insertar, recuperar, modificar y eliminar datos utilizando consultas SQL adecuadas.

Además, es esencial conocer y mitigar las vulnerabilidades de seguridad como la inyección SQL mediante el uso de consultas parametrizadas, que previenen ataques maliciosos y aseguran la integridad de los datos.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.



Fig.1. SQL injection.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Medidas preventivas"

https://on.soundcloud.com/xFPB2iB5qYtsAkoT7

/ 2. Base de datos objeto-relacional

En Java, una base de datos objeto-relacional es aquella que **combina características de las bases de datos relacionales y las bases de datos orientadas a objetos**, y que puede ser utilizada dentro del entorno de desarrollo de Java.

Estas bases de datos ofrecen la capacidad de almacenar y manipular objetos complejos de manera nativa, proporcionando una representación más fiel de las estructuras de datos utilizadas en la programación orientada a objetos.

En términos de Java, las bases de datos objeto-relacionales generalmente se utilizan a través de *frameworks* de mapeo objeto-relacional (ORM) como Hibernate, EclipseLink o JPA (Java Persistence API).

¡! Sabías que...

Estos frameworks proporcionan una capa de abstracción entre el código de Java y la base de datos subyacente, de modo que permiten a los desarrolladores interactuar con la base de datos utilizando objetos Java en lugar de escribir consultas SQL directamente.

Algunas **características** importantes de las bases de datos objeto-relacionales en Java son:

- **Mapeo objeto-relacional (ORM)**: Utilizan técnicas de mapeo para asociar clases y objetos de Java con las tablas y registros de la base de datos, lo que permite manipular los datos de forma transparente utilizando objetos Java.
- **Tipos de datos complejos**: Permiten el uso de tipos de datos complejos como objetos, *arrays* y colecciones, lo que facilita la representación de estructuras de datos complejas utilizadas en la programación orientada a objetos.
- **Herencia y polimorfismo**: Pueden representar relaciones de herencia entre clases Java empleando tablas de la base de datos, lo que hace posible definir jerarquías de clases y compartir atributos y métodos entre ellas.
- **Consultas de objetos**: Permiten realizar consultas usando el lenguaje de consulta de la base de datos (por ejemplo, SQL) o utilizando consultas orientadas a objetos que manipulan objetos Java en lugar de datos de la base de datos directamente.

Se podría decir, por tanto, que una base de datos objeto-relacional en Java combina las capacidades de las bases de datos relacionales y las bases de datos orientadas a objetos, y que puede ser utilizada dentro del entorno de desarrollo de Java a través de frameworks ORM. **Estas bases de datos ofrecen una representación más rica y flexible de los datos**, permitiendo a los desarrolladores trabajar con objetos Java de manera nativa.

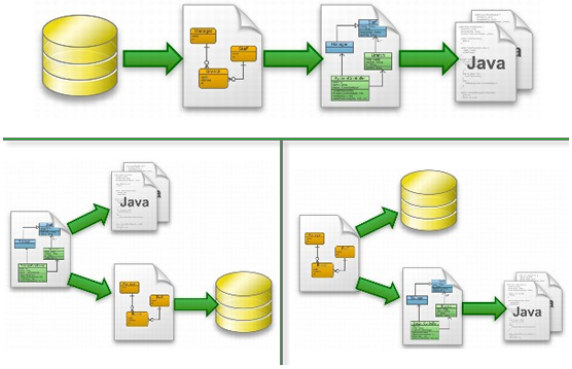


Fig.2. Modelo objeto-relacional BD.

En el siguiente audio podrás aprender más sobre el apartado anterior.

Audio 1. "Diferencias entre Hibernate, EclipseLink o JPA"
https://on.soundcloud.com/1Vfgnr1Zy885ybAq7

/ 3. ORM

Un ORM (*Object-Relational Mapping*) es una **técnica de programación** que permite convertir datos entre el sistema de tipos utilizado en un lenguaje de programación orientado a objetos y la representación de esos datos en una base de datos relacional.

En otras palabras, un ORM proporciona una capa de abstracción que permite a los desarrolladores interactuar con la base de datos utilizando objetos y clases en lugar de escribir consultas SQL directamente. La siguiente figura plasma la funcionalidad.

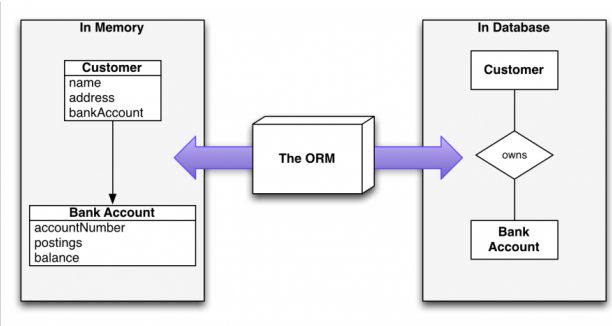


Fig.3. Mapeo objeto-relacional.

Los principales **componentes** de un ORM son los siguientes:

- **Mapeo objeto-relacional (*ORM Mapping*)**: Define la correspondencia entre las clases y los objetos en el lenguaje de programación orientado a objetos y las tablas y relaciones en la base de datos relacional. Cada clase en el modelo de objetos se asigna a una tabla en la base de datos, y cada atributo de la clase se asigna a una columna en esa tabla. Las relaciones entre clases (por ejemplo, relaciones uno a uno, uno a muchos, muchos a muchos) se mapean a relaciones entre tablas utilizando claves primarias y foráneas.

- **API de consulta (*Query API*)**: Este componente proporciona una API de consulta que permite a los desarrolladores realizar consultas sobre la base de datos utilizando objetos y clases en lugar de escribir consultas SQL directamente. La API de consulta generalmente incluye métodos para efectuar operaciones **CRUD** (crear, leer, actualizar, borrar) sobre los objetos del modelo de objetos y para filtrar, ordenar y agrupar los resultados de las consultas.

- **Administración de transacciones (*Transaction Management*)**: Los ORM suelen ofrecer funcionalidades para gestionar transacciones en la base de datos. Esto incluye iniciar, confirmar y revertir transacciones, así como definir el nivel de aislamiento y duración de las transacciones.

- **Mapeo de herencia (*Inheritance Mapping*)**: Algunos ORM proporcionan soporte para mapear jerarquías de herencia en el modelo de objetos a tablas en la base de datos. Esto permite representar relaciones de herencia entre clases y objetos utilizando tablas y relaciones en la base de datos relacional.

- **Caché de primer nivel (*First Level Cache*)**: Muchos ORM mantienen una caché de primer nivel para mejorar el rendimiento de las operaciones de lectura y escritura.

/ 4. Caso práctico 1: "¿Cuándo emplear Hibernate?"

Planteamiento: Utilizar un ORM como Hibernate facilita el desarrollo al proporcionar una capa de abstracción entre el modelo de datos relacional y el modelo de objetos en Java. Esto mejora la productividad, la mantenibilidad del código y la flexibilidad del sistema, lo que lo hace adecuado para aplicaciones que manejan datos complejos y relaciones en una base de datos relacional.

Nudo: Se está desarrollando un sistema de gestión de estudiantes para un centro. El mismo debe manejar información sobre estudiantes. ¿Cómo plantearías el desarrollo del sistema? ¿Sería conveniente emplear un ORM como Hibernate o valdría para este sistema una implementación directa con SQL mediante el uso de consultas SQL nativas?

Desenlace: A primera vista, el desarrollo con Hibernate proporciona una solución elegante y manejable para las relaciones entre objetos y se podría decir, por tanto, que sí, se podría utilizar un ORM como Hibernate en este contexto, ya que existen distintos estudiantes.

Sin embargo, estos estudiantes no tienen relación con ningún otro objeto como pudiera ser un nuevo objeto 'profesor' donde cada estudiante pertenece a la clase de un profesor y cada profesor puede tener varios estudiantes.

Por lo tanto, una implementación directa con SQL puede ser más eficiente y sencilla para este sistema mediante el uso de consultas SQL nativas, ya que el sistema no pide la gestión de relación entre distintos objetos.

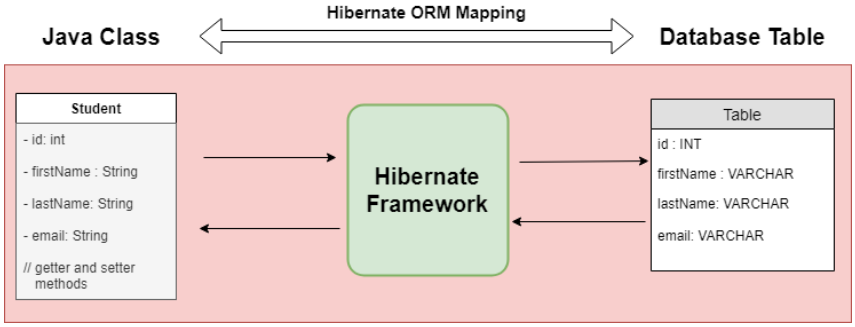


Fig.4. Hibernate ORM.

/ 5. CRUD

Las **operaciones CRUD** (crear, leer, actualizar, borrar) se emplean **para hacer referencia a las operaciones básicas realizadas en una base de datos**. Hasta ahora se ha trabajado con datos aislados en una base de datos relacional, pero, en un lenguaje de programación orientado a objetos como es Java, no es habitual trabajar de esta manera. Sin embargo, debido al extenso uso de los lenguajes orientados a objetos y el empleo de bases de datos relacionales, surge una técnica denominada mapeo objeto-relacional que consiste en vincular cada atributo de clase a un campo de la tabla. Es decir, convertir un objeto en un registro de la base de datos.

A cada clase que tengamos en la aplicación se le deberán añadir los siguientes **métodos**:

- **create()**: La operación de creación implica crear un nuevo objeto y almacenarlo en la base de datos mediante la sentencia INSERT. Para realizar esta operación, se crea un nuevo objeto en el lenguaje de programación, se establecen sus atributos y luego se persiste en la base de datos. Esta se encarga de asignar un identificador único al objeto y almacenarlo en la colección o tabla correspondiente.

- **read()**: La operación de lectura implica recuperar objetos almacenados en la base de datos mediante una sentencia SELECT y utilizarlos en la aplicación. En una base de datos objeto-relacional, es posible realizar consultas empleando filtros y criterios de búsqueda para recuperar objetos específicos que cumplan con ciertas condiciones. Los objetos recuperados se pueden usar en la aplicación de la misma manera que cualquier otro objeto en el lenguaje de programación que se esté utilizando.

- **update()**: La operación de actualización implica modificar los valores de los atributos de un objeto existente y persistir los cambios en la base de datos mediante la sentencia UPDATE. Para actualizar un objeto, primero se recupera el objeto existente, se modifican sus atributos y luego se guarda de nuevo en la base de datos.

- **delete()**: La operación de borrado implica eliminar un objeto existente de la base de datos mediante la sentencia DELETE. Se puede eliminar un objeto utilizando su identificador único o empleando otros criterios de búsqueda.

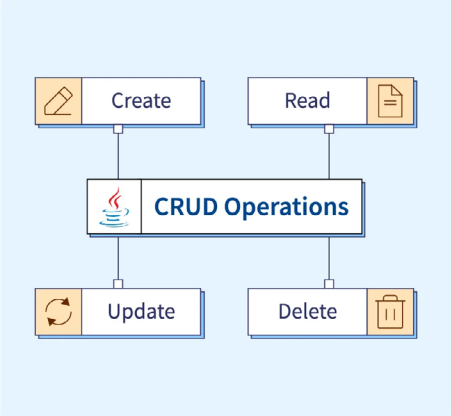


Fig.5. Operaciones CRUD.

/ 6. Objeto de acceso a datos: DAO

Hasta ahora, se ha manejado, dentro de las clases, cualquier operación CRUD. Por ejemplo, si partimos de una clase alumno con sus propiedades y métodos, se han creado dentro de la misma clase los métodos asociados a CRUD.

Desde el punto de la abstracción y siendo puristas, una clase debe plasmar sus características y comportamientos, siendo todo lo demás una distorsión de la abstracción.

Debido a esto, se genera un nuevo enfoque en el que cada clase se diseña, como hasta ahora, con sus atributos y métodos, y en el caso de que esa misma clase se necesite guardar en una BD, se implementará **una segunda clase denominada DAO con la finalidad de realizar las operaciones CRUD**.

Un DAO es un componente que proporciona una interfaz abstracta para interactuar con una fuente de datos, como una base de datos, un archivo o un servicio web. El propósito principal de un DAO es separar la lógica de acceso a datos de la lógica de negocio de una aplicación, lo que permite una mayor modularidad y facilita el mantenimiento y la prueba del código.

Vamos a verlo de manera práctica mediante el ejemplo de nuestra clase Alumno. Como se puede observar en la figura 6, la clase Alumno contiene los atributos y comportamientos propios de la clase y las operaciones CRUD.

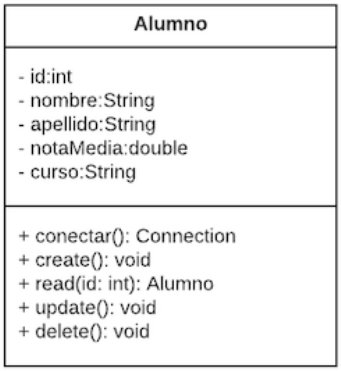


Fig.6. Clase alumno.

Sin embargo, mediante el uso de los objetos de acceso a datos, conseguimos desacoplar la lógica de negocio de la lógica de acceso a datos, como se puede observar en la figura 7.

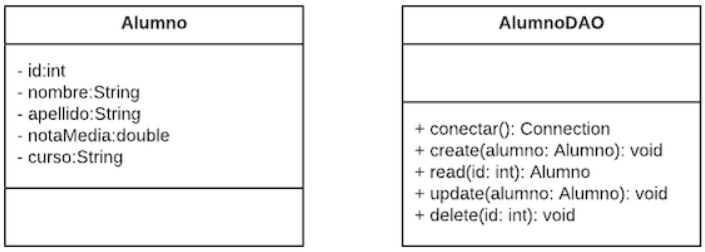


Fig.7. Clase Alumno y AlumnoDAO.

De esta manera, se consigue una mejor modularidad y mantenibilidad del código al proporcionar una capa de abstracción que permitirá una posible migración entre las diferentes fuentes de datos sin necesidad de afectar a la lógica de negocio.

/ 7. Ejecución de sentencias: CRUD

A continuación, se va a implementar la ejecución de las sentencias CRUD en la clase Alumno y AlumnoDAO previamente mostrada en la Figura 7.

7.1. Create()

Implementamos el método create() de la clase Alumno y AlumnoDAO siguiendo el diagrama de clases de las figuras 6 y 7 respectivamente.

```
public void create() {
    Connection con = conexion();

    String sql = "INSERT INTO Alumnos (num, nombre, apellido, media, curso) " +
                 "             VALUES ( ?,    ?,     ?,     ?,     ?  )";

    try {
        PreparedStatement sentencia = con.prepareStatement(sql);
        sentencia.setInt(1, id);
        sentencia.setString(2, nombre);
        sentencia.setDate(3, apellido);
        sentencia.setDouble(4, notaMedia);
        sentencia.setString(5, curso);

        sentencia.executeUpdate();

        con.close();
    } catch (SQLException ex) {
        System.out.println("Error al insertar.");
    }
}
```

Fig.8. Método create() de la clase Alumno.

```
public static void create(Alumno alumno) {
    if (alumno != null) {
        Connection conexion = conectar();

        String sql = "INSERT INTO Alumnos (num, nombre, apellido, media, curso) "
                + "             VALUES ( ?,    ?,     ?,     ?,     ?  )";

        try {
            PreparedStatement sentencia = conexion.prepareStatement(sql);
            sentencia.setInt(1, alumno.getId());
            sentencia.setString(2, alumno.getNombre());
            sentencia.setApellido(3, alumno.getApellido());
            sentencia.setDouble(4, alumno.getNotaMedia());
            sentencia.setString(5, alumno.getCurso());

            sentencia.executeUpdate();

            conexion.close();
        } catch (SQLException ex) {
            System.out.println("Error al insertar.");
        }
    }
}
```

Fig.9. Método create() de la clase AlumnoDAO.

7.2. Read()

Pasamos a implementar el método read() de la clase Alumno y AlumnoDAO siguiendo el diagrama de clases de las figuras 6 y 7 respectivamente.

```
static public Alumno read(int id) {
    Alumno alumno = null;
    String sql = "SELECT *"
               + "FROM Alumnos "
               + "WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);

        sentencia.setInt(1, id);

        ResultSet rs = sentencia.executeQuery();

        if (rs.next()) {
            String nombre = rs.getString("nombre");
            Date fNacimiento = rs.getString("apellido");
            Double notaMedia = rs.getDouble("media");
            String curso = rs.getString("curso");

            alumno = new Alumno(id, nombre, apellido, notaMedia, curso);
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }

    return alumno;
}
```

Fig.10. Método read() de la clase Alumno.

```
public static Alumno read(int id) {
    Alumno alumno = null;
    String sql = "SELECT *"
            + "FROM Alumnos "
            + "WHERE num = ?";
    try {
        Connection conexion = conectar();

        PreparedStatement sentencia = conexion.prepareStatement(sql);

        sentencia.setInt(1, id);

        ResultSet rs = sentencia.executeQuery();

        if (rs.next()) {
            String nombre = rs.getString("nombre");
            String apellido = rs.getApellido("apellido");
            Double notaMedia = rs.getDouble("media");
            String curso = rs.getString("curso");

            alumno = new Alumno(id, nombre, apellido, notaMedia, curso);
            conexion.close();
        }
    } catch (SQLException ex) {
        System.out.println("Error al consultar un alumno.");
    }

    return alumno;
}
```

Fig.11. Método read() de la clase AlumnoDAO.

7.3. Update()

Seguidamente, implementamos el método update() de la clase Alumno y AlumnoDAO siguiendo el diagrama de clases de las figuras 6 y 7 respectivamente.

```
public void update() {
    String sql = "UPDATE Alumnos "
                + "SET nombre=?, apellido=?, media=?, curso=? "
                + "WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);

        sentencia.setString(1, this.nombre);
        sentencia.setDate(2, this.apellido);
        sentencia.setDouble(3, this.notaMedia);
        sentencia.setString(4, this.curso);
        sentencia.setInt(5, id);

        sentencia.executeUpdate();
    } catch (SQLException ex) {
        System.out.println("Error al actualizar un alumno.");
    }
}
```

Fig.12. Método update() de la clase Alumno.

```java
public static void update(Alumno alumno) {
    if (alumno != null) {
        String sql = "UPDATE Alumnos "
                + "SET nombre=?, apellido=?, media=?, curso=? "
                + "WHERE num = ?";
        try {
            Connection conexion = conectar();
            PreparedStatement sentencia = conexion.prepareStatement(sql);

            sentencia.setString(1, alumno.getNombre());
            sentencia.setDate(2, alumno.getApellido();
            sentencia.setDouble(3, alumno.getNotaMedia());
            sentencia.setString(4, alumno.getCurso());
            sentencia.setInt(5, alumno.getId());

            sentencia.executeUpdate();
            conexion.close();
        } catch (SQLException ex) {
            System.out.println("Error al actualizar un alumno.");
        }
    }
}
```

Fig.13. Método update() de la clase AlumnoDAO.

7.4. Delete()

A continuación, implementamos el método delete() de la clase Alumno y AlumnoDAO siguiendo el diagrama de clases de las figuras 6 y 7 respectivamente.

```java
public void delete() {
    String sql = "DELETE FROM Alumnos "
                + "WHERE num = ?";
    try {
        Connection con = conexion();
        PreparedStatement sentencia = con.prepareStatement(sql);

        sentencia.setInt(1, id);

        sentencia.executeUpdate();
    } catch (SQLException ex) {
        System.out.println("Error al eliminar un alumno.");
    }
}
```

Fig.14. Método delete() de la clase Alumno.

```java
public static void delete(int id) {
    String sql = "DELETE FROM Alumnos "
            + "WHERE num = ?";
    try {
        Connection conexion = conectar();
        PreparedStatement sentencia = conexion.prepareStatement(sql);

        sentencia.setInt(1, id);

        sentencia.executeUpdate();
        conexion.close();
    } catch (SQLException ex) {
        System.out.println("Error al eliminar un alumno.");
    }
}
```

Fig.15. Método delete() de la clase AlumnoDAO.

En el siguiente recurso podrás aprender más sobre el apartado anterior.

Recuerda...

Para comprobar que la aplicación está realizando las operaciones oportunas, es necesario acceder al SGBD, ya sea a través de un terminal o de algún cliente gráfico, y revisar las operaciones.

8. Movimiento y posicionamiento del cursor ResultSet

En el tema anterior, se explicaron la clase y los distintos tipos de la clase ResultSet. En esta sección, se van a exponer los métodos que mueven y posicionan el cursor.

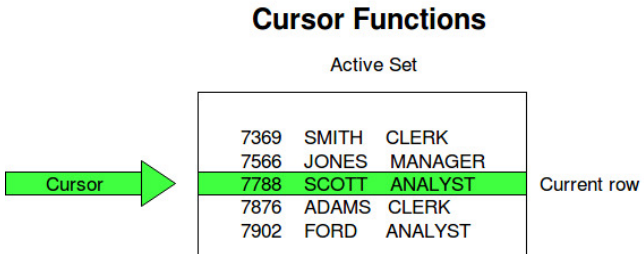


Fig.16. Posición del cursor.

8.1. Movimiento del cursor

Para mover el cursor de un ResultSet en Java, podemos utilizar varios **métodos proporcionados por la interfaz ResultSet**. Aquí presentamos algunos de los métodos más comunes para mover el cursor hacia delante y hacia atrás a través de los resultados:

- **next()**: Mueve el cursor al siguiente registro en el conjunto de resultados. Retorna 'true' si hay un registro siguiente y 'false' si el cursor está después del último registro.
- **previous()**: Traslada el cursor al registro anterior en el conjunto de resultados. Esto solo es posible si el ResultSet es del tipo ResultSet.TYPE_SCROLL_SENSITIVE o ResultSet.TYPE_SCROLL_INSENSITIVE.
- **first()**: Mueve el cursor al primer registro en el conjunto de resultados.
- **last()**: Lleva el cursor hasta el último registro en el conjunto de resultados.
- **absolute(int row)**: Mueve el cursor a una posición absoluta dada en el conjunto de resultados.
- **relative(int rows)**: Traslada el cursor a una posición relativa dada en relación con la posición actual del cursor.
- **beforeFirst()**: Mueve el cursor a antes del primer registro, justo antes de la primera fila. Esta es una posición ficticia y no contiene datos.
- **afterLast()**: Lleva el cursor hasta después del último registro, justo después de la última fila. Esta es una posición ficticia y no contiene datos.

El método adecuado a utilizar dependerá de los requisitos específicos de nuestra aplicación y de cómo deseemos recorrer los resultados obtenidos de la consulta SQL.

8.2. Ubicación del cursor

Al mover el cursor a lo largo de la tabla, se puede dar el caso que no sepamos en qué posición se encuentra el mismo y que sea necesario **preguntar la posición**. A continuación se muestran los métodos más comunes:

- **boolean isBeforeFirst()**: Indica si el cursor está colocado justo delante de la primera fila.
- **boolean isAfterLast()**: Señala si el cursor está ubicado justo detrás de la última fila.
- **boolean isFirst()**: Devuelve 'true' si el cursor apunta a la primera fila.
- **boolean isLast ()**: Devuelve 'true' si el cursor apunta a la última fila.

/ 9. SQL injection

La inyección SQL **es una vulnerabilidad de seguridad** que amenaza la integridad y confidencialidad de los datos en las aplicaciones web y de bases de datos. En el contexto de Java, la inyección SQL ocurre cuando los datos proporcionados por el usuario no se validan adecuadamente antes de ser utilizados en consultas SQL dinámicas, lo que permite a los atacantes ejecutar comandos SQL no autorizados.

La inyección SQL se produce principalmente en sistemas que usan consultas SQL dinámicas construidas concatenando cadenas de texto con datos proporcionados por el usuario. Esta práctica, si no se maneja correctamente, puede abrir la puerta a ataques maliciosos.

Un ejemplo clásico de inyección SQL en Java es el que se da cuando se utiliza una consulta SQL dinámica para buscar usuarios en una base de datos, como se muestra en el código de la figura 17.

```
String nombreUsuario = request.getParameter("nombreUsuario");
String consulta = "SELECT * FROM usuarios WHERE nombre = '" + nombreUsuario + "'";
```

Fig.17. Consulta SQL vulnerable.

En este caso, si un atacante ingresa un valor como ' OR '1'='1, la consulta resultante sería:

```
SELECT * FROM usuarios WHERE nombre = '' OR '1'='1'
```

Esta consulta siempre devolverá todos los registros de la tabla de usuarios, lo que podría permitir al atacante acceder a información confidencial o ejecutar acciones no autorizadas.

Para mitigar la inyección SQL en Java, es fundamental utilizar **consultas parametrizadas o preparadas**, en lugar de concatenar cadenas de texto para construir consultas SQL. Al utilizar **PreparedStatement**, los parámetros proporcionados por el usuario se tratan como datos y no como parte de la consulta SQL, lo que previene la ejecución de comandos maliciosos.

Además, es esencial **validar y filtrar los datos de entrada del usuario** para garantizar que cumplan con los requisitos esperados, antes de utilizarlos en consultas SQL. Esto puede incluir la verificación de tipos de datos, la longitud de cadenas y la eliminación de caracteres especiales que podrían ser utilizados en ataques de inyección SQL.

La inyección SQL es una grave amenaza para la seguridad de las aplicaciones Java, pero puede mitigarse mediante la adopción de buenas prácticas de codificación, como el uso de consultas parametrizadas y la validación de datos de entrada del usuario. Al tomar medidas proactivas para prevenir la inyección SQL, los desarrolladores pueden proteger eficazmente sus aplicaciones y datos contra posibles ataques maliciosos.

¡! Sabías que...

Uno de los primeros y más famosos ataques de inyección SQL fue descubierto en un sitio web gubernamental del Reino Unido en el año 2000. Su propósito era acceder ilegalmente a bases de datos sensibles, lo que reveló la vulnerabilidad de muchos sitios web.

/ 10. Sentencias parametrizadas

La seguridad de las aplicaciones Java es un aspecto crucial en el desarrollo de *software* moderno. La inyección SQL, una de las vulnerabilidades más comunes, representa una amenaza significativa para la integridad y la confidencialidad de los datos en las aplicaciones que interactúan con bases de datos. Sin embargo, Java ofrece un mecanismo poderoso para mitigar esta amenaza: **las sentencias parametrizadas**.

Las sentencias parametrizadas constituyen una **técnica de seguridad** que permite construir consultas SQL de manera dinámica utilizando marcadores de posición para los datos proporcionados por el usuario. En lugar de concatenar cadenas de texto con datos no validados, como se hace en las consultas SQL dinámicas tradicionales, las sentencias parametrizadas permiten **separar los datos de la estructura de la consulta**, lo que reduce significativamente el riesgo de inyección SQL.

En Java, las sentencias parametrizadas se implementan utilizando la interfaz PreparedStatement. Estas sentencias permiten definir consultas SQL con parámetros marcados con un signo de interrogación (?) en lugar de los valores reales. Luego, los valores de los parámetros se establecen de forma segura empleando métodos específicos proporcionados por PreparedStatement, como setString(), setInt(), setBoolean(), entre otros.

A continuación se muestra, en el código de la figura 18, un ejemplo de cómo utilizar sentencias parametrizadas en Java para evitar la inyección SQL:

```
String nombreUsuario = request.getParameter("nombreUsuario");
String consulta = "SELECT * FROM usuarios WHERE nombre = ?";
try (Connection conn = DriverManager.getConnection(url, user, password);
     PreparedStatement pstmt = conn.prepareStatement(consulta)) {
    pstmt.setString(1, nombreUsuario);
    ResultSet resultSet = pstmt.executeQuery();
    // Procesar los resultados
} catch (SQLException e) {
    // Manejar cualquier excepción
}
```

Fig.18. Sentencia parametrizada.

En este ejemplo, la consulta SQL utiliza un marcador de posición (?) para el nombre de usuario, en lugar de concatenar directamente la cadena proporcionada por el usuario. Luego, se emplea el método setString() de PreparedStatement para establecer de forma segura el valor del parámetro.

Cuando se utilizan sentencias parametrizadas en Java, se reduce drásticamente el riesgo de inyección SQL al evitar la concatenación directa de datos proporcionados por el usuario en las consultas SQL. Esto fortalece la seguridad de las aplicaciones Java y protege los datos sensibles de posibles ataques maliciosos.

/ 11. Caso práctico 2: "Inyección SQL"

Planteamiento: La inyección SQL es crítica, porque permite a los atacantes manipular bases de datos mediante entradas maliciosas. Esto puede producir pérdida de datos, acceso no autorizado y compromiso de la integridad del sistema, amenazando la seguridad y funcionalidad de aplicaciones web y sistemas empresariales.

Nudo:

```
'; DROP TABLE Products; --
```

Fig.19. Texto introducido en el buscador.

Se está realizando el desarrollo de una aplicación web simple que permite a los usuarios buscar productos por nombre en una base de datos. El sistema utiliza JDBC para interactuar con la base de datos MySQL y, a la hora de realizar una búsqueda, un usuario introduce el texto que se muestra en la figura 19. ¿Qué resultado obtiene al insertar dicho código?

Desenlace:

```
SELECT * FROM Products WHERE name = ''; DROP TABLE Products; --
```

Fig.20. Consulta SQL.

La explicación del siguiente código es la siguiente:

- El usuario ingresa ' para cerrar la declaración SQL que se está construyendo en el código de la aplicación.
- DROP TABLE Products; es una instrucción SQL válida que eliminaría la tabla Products de la base de datos si se ejecuta.
- -- es un comentario en SQL para asegurar que el resto de la consulta original se ignore.

Por tanto, el resultado de la búsqueda del siguiente código, en el caso de que la base de datos no esté protegido contra inyecciones SQL, sería una consulta SQL que se verá como muestra la figura 20.

Como consecuencia, la tabla Products se eliminaría de la base de datos, causando una pérdida permanente de datos y posiblemente una interrupción en el funcionamiento de la aplicación.

/ 12. Bases de datos e interfaz gráfica

El trabajar con una interfaz gráfica es algo fundamental hoy en día, ya que no vamos a realizar un programa de gestión y lo vamos a entregar en consola, eso sería impensable.

Generalmente, **el uso de la interfaz gráfica va unido al de las bases de datos**, puesto que conseguiremos que todo sea mucho más visual y fácil de entender para el usuario.

Igual que vimos en la unidad de interfaces gráficas, la información, la introduciremos mediante cajas de texto, tendremos botones que nos permitirán conectarnos a la base de datos, desconectarnos, ejecutar una consulta, etc.

Quizás lo más complicado sea el mostrar los datos de una consulta SELECT, ya que tendremos que utilizar una tabla, y ya vimos que las tablas son un poco más complicadas de utilizar que los demás componentes.

Haciendo un poco de repaso, para rellenar una tabla, *grosso modo*, necesitamos obtener su modelo, introducir los datos en un array y luego ir agregando esos datos en filas.

Estos pasos son exactamente los mismos para el caso de utilizar una base de datos.

Aquí podremos emplear el Modelo Vista-Controlador para terminar de dejar a la perfección los programas.

Imaginemos que tenemos una tabla para mostrar los datos de personas y queremos rellenar su nombre, apellidos y teléfono, el código para obtener esos datos y rellenar la tabla podría ser el que se muestra en la figura 21.

```
// Introducimos los datos en la tabla
DefaultTableModel modelo = (DefaultTableModel) tablaDatos.getModel();
for(Persona per : personas)
{
    ArrayList<String> fila = new ArrayList<>();
    fila.add(per.getNombre());
    fila.add(per.getApellidos());
    fila.add(per.getTelefono());

    modelo.addRow(fila.toArray());
}
```

Fig.21. Rellenando una tabla.

Podemos ver cómo obtenemos el modelo de la tabla y recorremos un array llamado 'personas' que está compuesto de objetos de tipo Persona. Al recorrerlo, obtenemos los datos necesarios de las personas y los almacenamos en otro array que será la fila, agregándolo al modelo en último lugar.



Vídeo 1. "Ejemplo completo de acceso con interfaz gráfica"

https://bit.ly/39kPbSW



/ 13. Modelo-Vista-Controlador

El patrón Modelo-Vista-Controlador (MVC) es un **patrón arquitectónico que separa la lógica de la aplicación en tres componentes principales: Modelo, Vista y Controlador**. Este enfoque promueve la modularidad y la escalabilidad del código, facilitando el desarrollo, la prueba y el mantenimiento de las aplicaciones.

- **Modelo**: Representa los datos y la lógica de negocio de la aplicación. Generalmente, consiste en clases Java que encapsulan los datos y proporcionan métodos para acceder y manipular esos datos. Estas clases pueden ser simples POJO (*Plain Old Java Objects*) o pueden contener lógica más compleja, como la interacción con una base de datos o un servicio web.

- **Vista**: Es la interfaz de usuario de la aplicación, que presenta los datos al usuario y permite la interacción con la aplicación. Habitualmente, consiste en clases que representan componentes de interfaz de usuario, como ventanas, botones, campos de texto, etc. Estas clases pueden implementarse utilizando bibliotecas de UI como Swing, JavaFX o frameworks web como Servlets, JSP, JSF, etc.

- **Controlador**: Actúa como intermediario entre el modelo y la vista, maneja las solicitudes del usuario y coordina las interacciones entre el modelo y la vista. Generalmente, consiste en clases que responden a eventos del usuario, como clics de botón o entradas de teclado, y que llaman a métodos en el modelo para actualizar los datos según sea necesario. Estas clases pueden implementarse utilizando ActionListener, en el caso de aplicaciones de escritorio Swing, o Servlets o controladores, en el caso de aplicaciones web.

13.1. Interacción MVC

Los componentes del patrón de diseño MVC interactúan de la manera que se muestra en la figura 22.

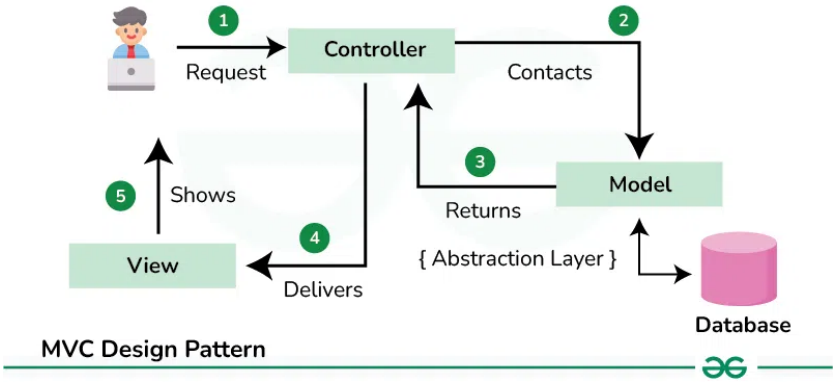


Fig.22. Aplicación del patrón MVC.

- El controlador recibe las solicitudes del usuario y llama a métodos en el modelo para realizar operaciones en los datos.
- El modelo actualiza los datos según las operaciones solicitadas por el controlador.
- La vista consulta el modelo para obtener los datos actualizados y los presenta al usuario.
- La vista también puede enviar eventos al controlador para informar sobre las acciones del usuario.

/ 14. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema, se continúa con un nuevo tipo de bases de datos, las bases de datos orientadas relacionales, profundizando en una nueva técnica que permite separar la lógica de la aplicación y la lógica de la base de datos añadiendo una capa de abstracción.

Se ha visto cómo realizar las operaciones típicas de toda base de datos, como insertar, actualizar, borrar y consultar datos con diferentes criterios de búsqueda, añadiendo esta nueva capa de abstracción. Además, se han explicado las vulnerabilidades de la base de datos y cómo solucionar este problema mediante un nuevo tipo de sentencias.

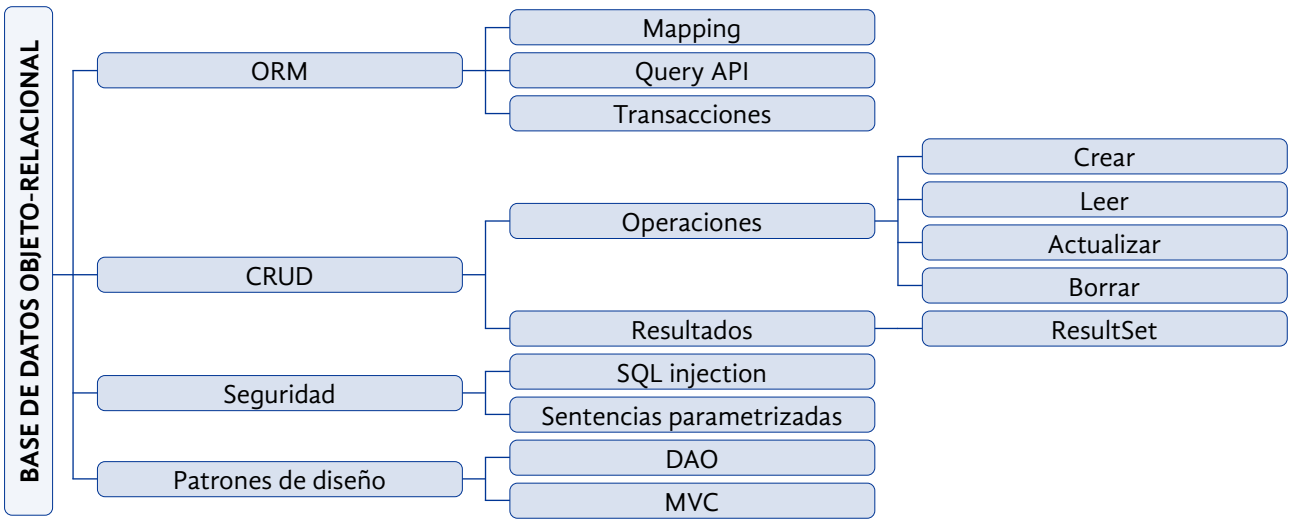


Fig.23. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Para evitar este tipo de ataques, es fundamental utilizar técnicas de protección como:

- **PreparedStatements**: Usar consultas preparadas en JDBC para separar claramente los datos de los comandos SQL.
- **ORM (Hibernate, JPA)**: Emplear un ORM puede mitigar automáticamente la mayoría de las vulnerabilidades de inyección SQL al manejar de manera segura las consultas a la base de datos.
- **Validación de entradas**: Validar las entradas de usuario para evitar caracteres especiales y comandos SQL en los campos de entrada.
- **Principio de Menor Privilegio**: Asegurarse de que las credenciales de conexión a la base de datos tengan los permisos mínimos necesarios para ejecutar las operaciones requeridas.

/ 15. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: *Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.